My PlayerList and SalaryList classes both have add, search, update, delete, and display functions. For a small Java student project, should I create a generic list class or keep PlayerList and SalaryList separate? Compare both approaches.

For your small Java student project, keep `PlayerList` **and** `SalaryList` **separate.**

A generic list class can reduce repeated CRUD code because both classes use `ArrayList` and have add, search, update, delete, and display functions. However, it becomes harder to understand because each data type still needs different rules:

- `PlayerList` checks duplicate player IDs and duplicate shirt numbers.
- `SalaryList` checks whether the related `playerId` exists and gets the player's calculated monthly salary.
- Player and Salary also have different input, update, and display logic.

With a generic class, you would need extra interfaces, abstract methods, or functional parameters to tell the generic class how to find IDs, validate fields, and display each object. That is useful in a larger system, but it may make a student console project harder to explain and debug.

Keeping separate classes has some duplicated structure, but the code is clearer:

Java

```
PlayerList
- addPlayer()
- findPlayerById()
- updatePlayer()
- deletePlayer()
- displayAllPlayers()

SalaryList
- addNewSalaryRecord()
- findSalaryById()
- updateSalaryRecord()
- deleteSalaryRecord()
- displayAllSalaryRecords()
```

This is especially suitable for your project because `SalaryList` depends on `PlayerList` to validate `playerId` and calculate the correct salary. Keep the menu flow similar in both classes, but allow each class to handle its own business rules.

```java
public class PlayerList {
    private ArrayList<Player> players;

    public PlayerList() {
        players = new ArrayList<>();
    }
```

```java
public class SalaryList {
    private ArrayList<Salary> salaryRecords;

    public SalaryList() {
        salaryRecords = new ArrayList<>();
    }
```